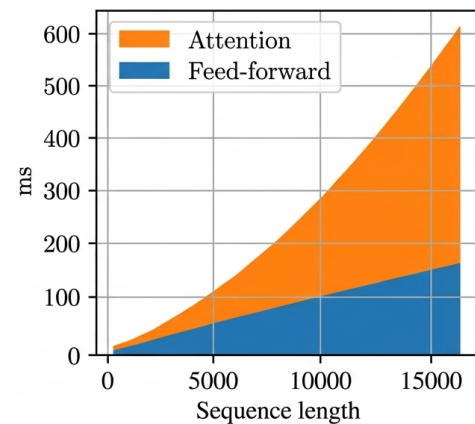
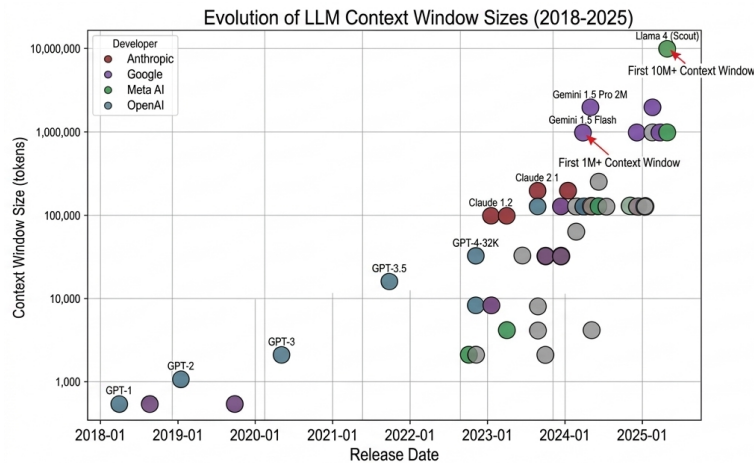


Lecture 4: Attention and Mixture of Experts

Attention Is All You Need

Cost of attention rises with large context sizes ... how do we control those costs?



- Combine local + global attention.
- Systems engineering.

But what if we want more radical and potentially large gains?

Linear attention

Consider the normal attention operation .. $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{n \times d_k}$ and $V \in \mathbb{R}^{n \times d_v}$..

$$\text{Attn}(Q, K, V) = \rho(QK^T)V.$$

This is quadratic due to QK^T ($n^2 d_k$). Can we do better (when ρ is the identity)?

$$(QK^T)V = Q(K^T V).$$

This is very silly, but surprisingly important .. we get from $n^2 d_k + n^2 d_v$ to $2nd_k d_v$.

Recurrent form of linear attention

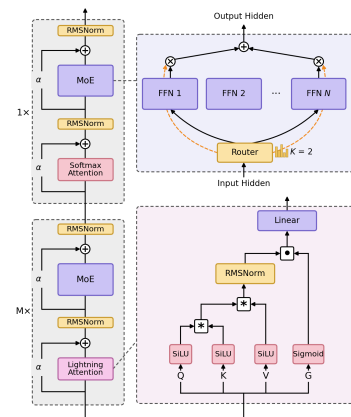
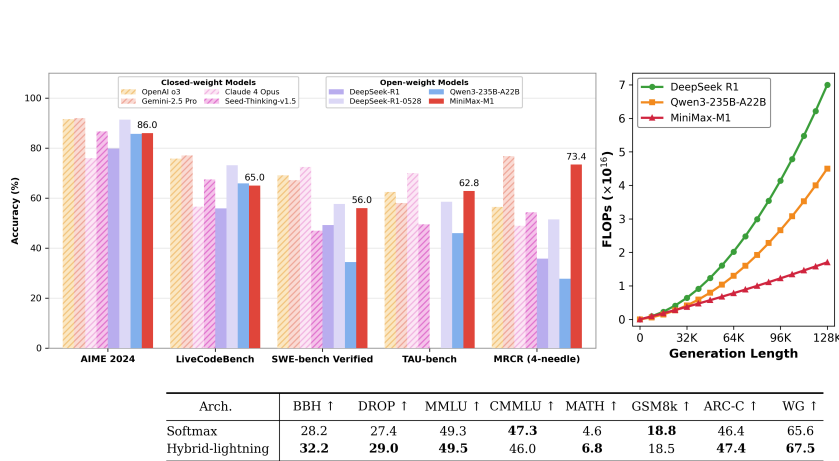
This is linear time (great) but the even nicer thing is that this looks like a RNN ...

$$S_t = S_{t-1} + k_t v_t^T \text{ and } y_t = q_t^T S_t.$$

This "duality" allows us to train efficiently (using the parallel, quadratic form) and inference efficiently (using the serial, linear form).

Note: if one weights S_{t-1} by γ , you get RetNet.

Minimax M1



Remark: Minimax M1 (and minimax-text-01) use a 7-to-1 hybrid (7 linear, 1 full) linear attention.

Performance (generally) seems strong, and they show linear scaling in context length.

From linear attention to Mamba-2

Let's generalize linear attention a little bit and add per-position weights ..

$$S_t = \gamma_t S_{t-1} + k_t v_t^T \text{ and } y_t = q_t^T S_t + v_t^T D \text{ with } \gamma_t = f(x_t).$$

There is a lot more words to justify this (go read the Mamba-2 paper) but the mechanics is that we can make linear attention more expressive via gating (gating is good)!

Remark: This continues to have duality properties (compute γ in parallel, apply duality).

Nemotron 3

Mamba attention hybrid (3-1 ish) - comparable (or better) performance to other similar models.

Gated delta net (and friends)

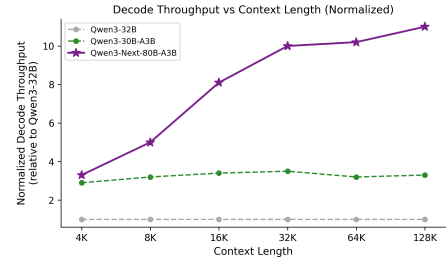
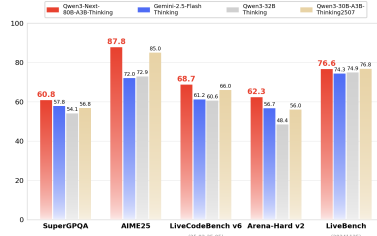
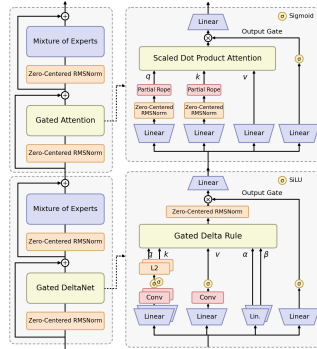
Let's generalize things further - gate the input and selectively erase state ..

$$S_t = \gamma_t(I - \beta_t k_t k_t^T) S_{t-1} + \beta_t k_t v_t^T \text{ and } y_t = q_t^T S_t \text{ with } \gamma_t = f(x_t) \text{ and } \beta_t = f(x_t).$$

The gated delta net adds a "no input operation" gate ($\beta = 0$) and *erases* anything in the direction of the current key ($I - \beta_t k_t k_t^T$).

Remark: Close relationships to various fast weight programming / test time training ideas.

Qwen 3.5 / Next



The newest Qwen are 3-1 GDN / attention hybrids. Once again, pretty reasonable performance with good inference characteristics.

Not many controlled ablations, but some evidence of low losses at small hybrid scales.

Alternative to hybrids: sparse adaption

Instead of attending to every token, do sparse attention (DSA)

Prototype of DSA. The prototype of DSA (DeepSeek Attention) primarily consists of two components: a lightning indexer and a fine-grained token selection mechanism.

The **lightning indexer** computes the index score $I_{t,s}$ between the query token $h_t \in \mathbb{R}^d$ and a preceding token $h_s \in \mathbb{R}^d$, determining which tokens to be selected by the query token:

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \text{ReLU}(q_{t,j}^I \cdot k_s^I),$$

where H^I denotes the number of indexer heads; $q_{t,j}^I \in \mathbb{R}^{d^I}$ and $w_{t,j}^I \in \mathbb{R}$ are derived from the query token h_t and $k_s^I \in \mathbb{R}^{d^I}$ is derived from the preceding token h_s

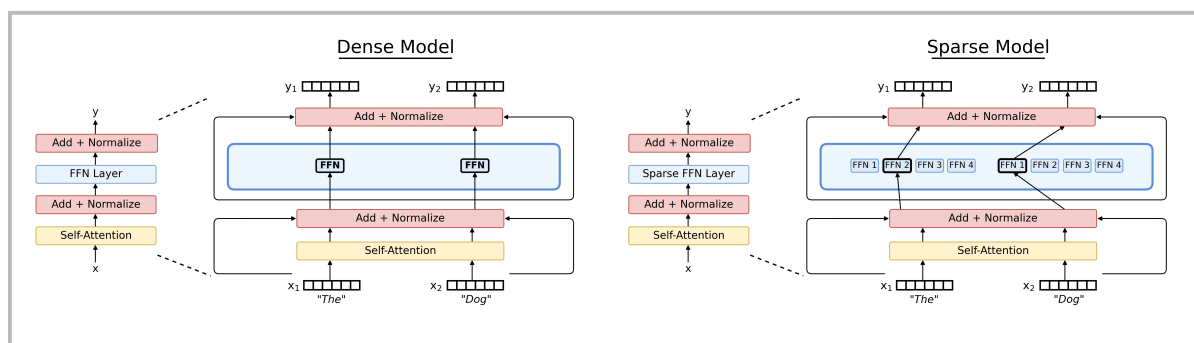
Remark: We choose the ReLU as the activation function for throughput considerations. Given the lightning indexer has a small number of heads and can be implemented in FP8, its computational efficiency is remarkable.

Given the indexer scores $\{I_{t,s}\}$ for each query token h_t , the **token selection mechanism** retrieves only the key-value entries $\{c_s\}$ corresponding to the top- k index scores. Then, the attention output u_t is computed by applying the attention mechanism between the query token h_t and the sparsely selected key-value entries $\{c_s\}$:

$$u_t = \text{Attn}(h_t, \{c_s \mid I_{t,s} \in \text{Top-}k(I_t)\}).$$

Remark: The indexer can be very lightweight, giving significant gains and can be "post-hoc" adapted after dense short context pretraining.

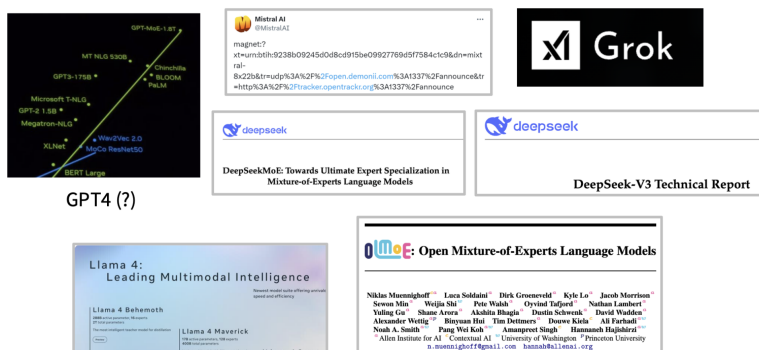
Mixture of Experts



[Fedus et al 2022]

What's a MoE?

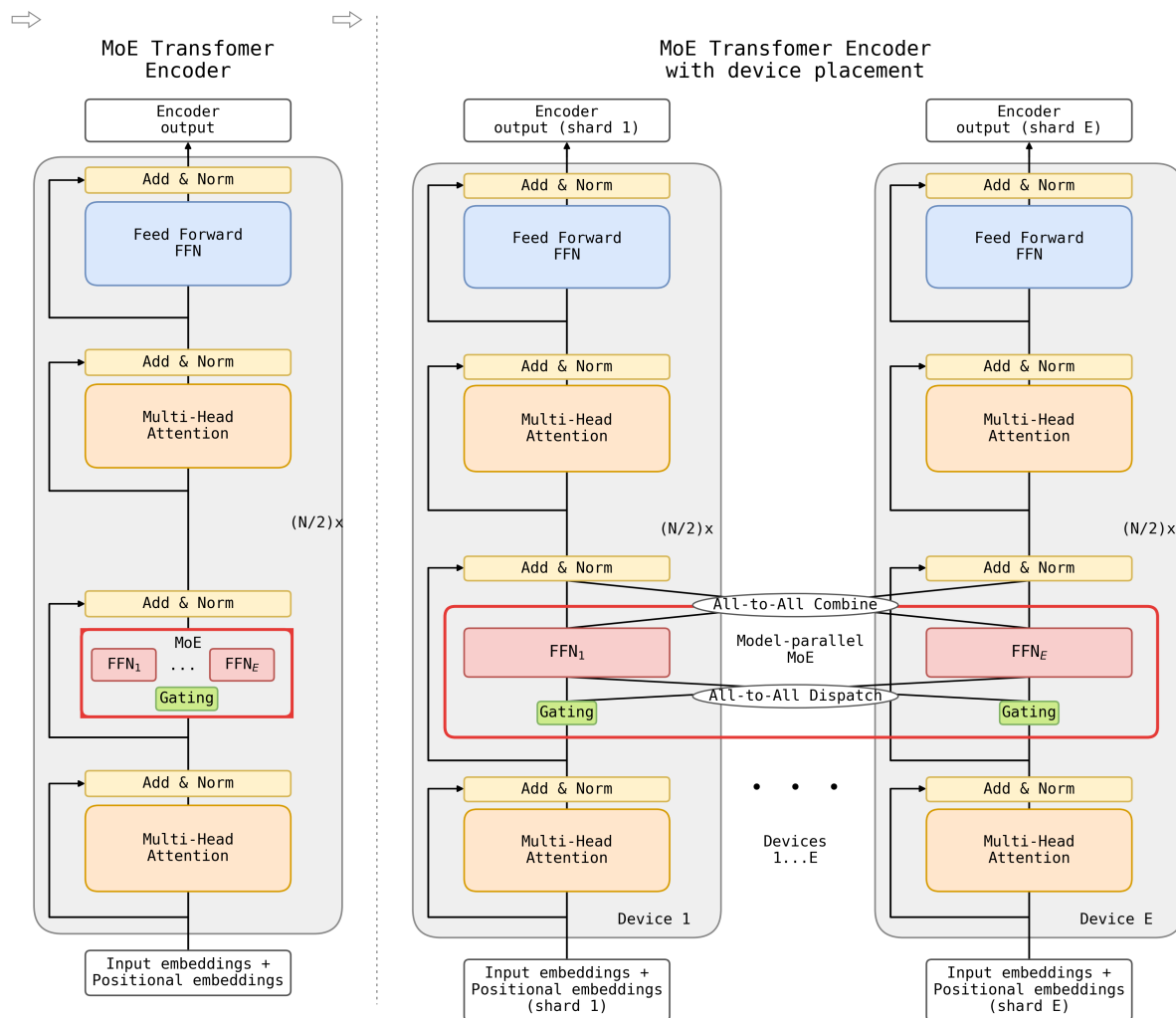
Replace big feedforward with (many) big feedforward networks and a selector layer.



Claim: You can increase the # experts without affecting FLOPs.

Why are MoEs getting popular?

- Same FLOPs, more parameters does better.
- Faster to train MoEs.
- **Highly competitive vs dense equivalents**
- Parallelizable to many devices.



Some MoE results - from the West

- MoEs are most of the highest-performance open models and are quite quick.
- **Examples:** LLaMA 4 Maverick, Gemini 2.0 Flash, GPT-4o.

Earlier MoE results from Chinese groups - Qwen

Chinese LLM companies are also doing quite a bit of MoE work on the smaller end.

Model	MMLU	GSM8K	HumanEval	Multilingual	MT-Bench
Mistral-7B	64.1	47.5	27.4	40.0	7.60
Gemma-7B	64.6	50.9	32.3	-	-
Qwen1.5-7B	61.0	62.5	36.0	45.2	7.60
DeepSeekMoE 16B	45.0	18.8	26.8	-	6.93
Qwen1.5-MoE-A2.7B	62.5	61.5	34.2	40.8	7.17

Model	#Parameters	#{(Activated) Parameters}
Mistral-7B	7.2	7.2
Qwen1.5-7B	7.7	7.7
Gemma-7B	8.5	7.8
DeepSeekMoE 16B	16.4	2.8
Qwen1.5-MoE-A2.7B	14.3	2.7

Earlier MoE results from Chinese groups - DeepSeek

There's also some good ablation work on MoEs showing they're generally good.

Metric	# Shot	Dense	Hash Layer	Switch
# Total Params	N/A	0.2B	2.0B	2.0B
# Activated Params	N/A	0.2B	0.2B	0.2B
FLOPs per 2K Tokens	N/A	2.9T	2.9T	2.9T
# Training Tokens	N/A	100B	100B	100B
Pile (Loss)	N/A	2.060	1.932	1.881
HellaSwag (Acc.)	0-shot	38.8	46.2	49.1
PIQA (Acc.)	0-shot	66.8	68.4	70.5
ARC-easy (Acc.)	0-shot	41.0	45.3	45.9
ARC-challenge (Acc.)	0-shot	26.0	28.2	30.2
RACE-middle (Acc.)	5-shot	38.8	38.8	43.6
RACE-high (Acc.)	5-shot	29.0	30.0	30.9
HumanEval (Pass@1)	0-shot	0.0	1.2	2.4
MBPP (Pass@1)	3-shot	0.2	0.6	0.4
TriviaQA (EM)	5-shot	4.9	6.5	8.9
NaturalQuestions (EM)	5-shot	1.4	1.4	2.5

Why haven't MoEs been more popular?

Infrastructure is complex / advantages on multi-node.

At a high level, sparsity is good when you have accelerators (e.g GPU/TPU) to host all additional parameters that comes when using sparsity. Typically models are trained using data-parallelism where different machines will get different slices of the training/inference data. The machines used for operating on the different slices of the data can now be used to host many more model parameters. Therefore, sparse models are good when training with data-parallelism and/or have high throughput while serving: training/serving on many machines, which can host all of the parameters. [Fedus et al 2022]

Training objectives are somewhat heuristic (and somewhat unstable).

What MoEs generally look like?

- Typical: replace the MLP with MoE layer.
- Less common: MoEs for attention heads.

Routing function - overview

Many of the routing algorithms boils down to "choose top k".

		Tokens		
		T1	T2	T3
Experts	E1	3.13	0.14	0.74
	E2	0.51	-0.25	1.58
	E3	-1.32	1.97	0.1
	E4	2.25	2.61	0.02
	E5	-2.81	-0.68	-0.41

Token chooses expert

		Tokens		
		T1	T2	T3
Experts	E1	Choose Top-K		
	E2	0.51	-0.25	1.58
	E3	-1.32	1.97	0.1
	E4	2.25	2.61	0.02
	E5	-2.81	-0.68	-0.41

Expert chooses token

		Tokens		
		T1	T2	T3
Experts	E1	3.13	0.14	0.74
	E2	0.51	-0.25	1.58
	E3	-1.32	1.97	0.1
	E4	2.25	2.61	0.02
	E5	-2.81	-0.68	-0.41

Global routing via optimization

Common routing methods

Almost all the MoEs do a standard "token choice top- k " routing.

- Top- k : Used in *most* MoEs.
- **Examples:** Switch transformer ($k = 1$), Gshard ($k = 2$), Grok ($k = 2$), Mixtral ($k = 2$), Qwen ($k = 4$), DBRX ($k = 4$), DeepSeek ($k = 7$).
- Hashing: Used to baseline MoEs.

Other routing methods

RL to learn routes.

- Used in some of the earliest work
- Bengio 2013, not common now

Solve a matching problem

- Linear assignment for routing
- Used in various papers like Clark 2022

Top-k routing in detail

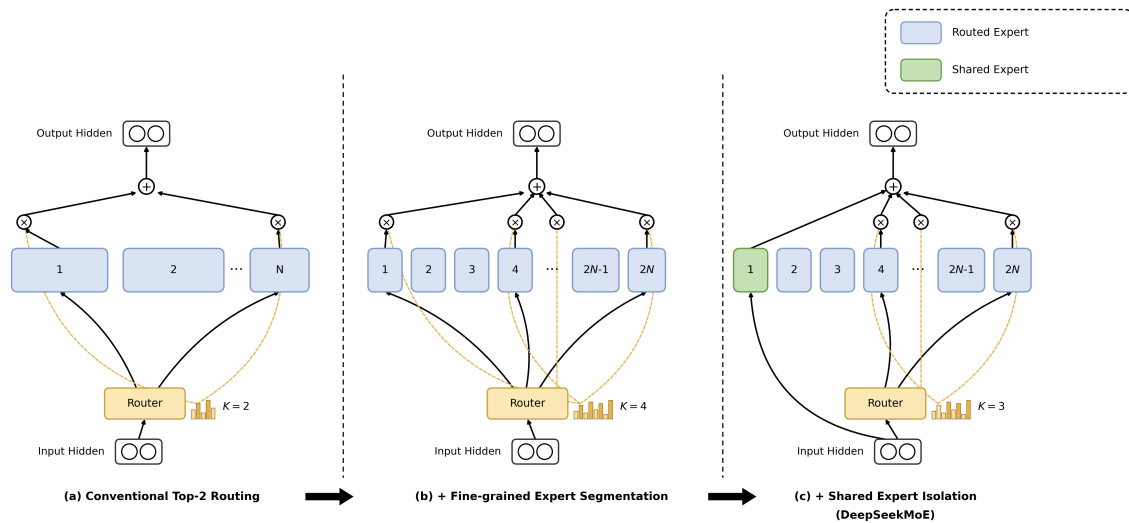
Most papers do the old and classic top-k routing. How does it work?

- This is the DeepSeek (v1-2) router (Grok, Qwen do this too).

$$\begin{aligned}
 \mathbf{h}_t^l &= \sum_{i=1}^N \left(\overset{\text{Gating}}{\downarrow} g_{i,t} \text{FFN}_i(\mathbf{u}_t^l) \right) + \mathbf{u}_t^l, \\
 g_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise,} \end{cases} \\
 s_{i,t} &= \text{Softmax}_i(\mathbf{u}_t^{lT} \mathbf{e}_i^l), \\
 &\quad \uparrow \\
 &\quad \text{Gates selected by a logistic regressor}
 \end{aligned}$$

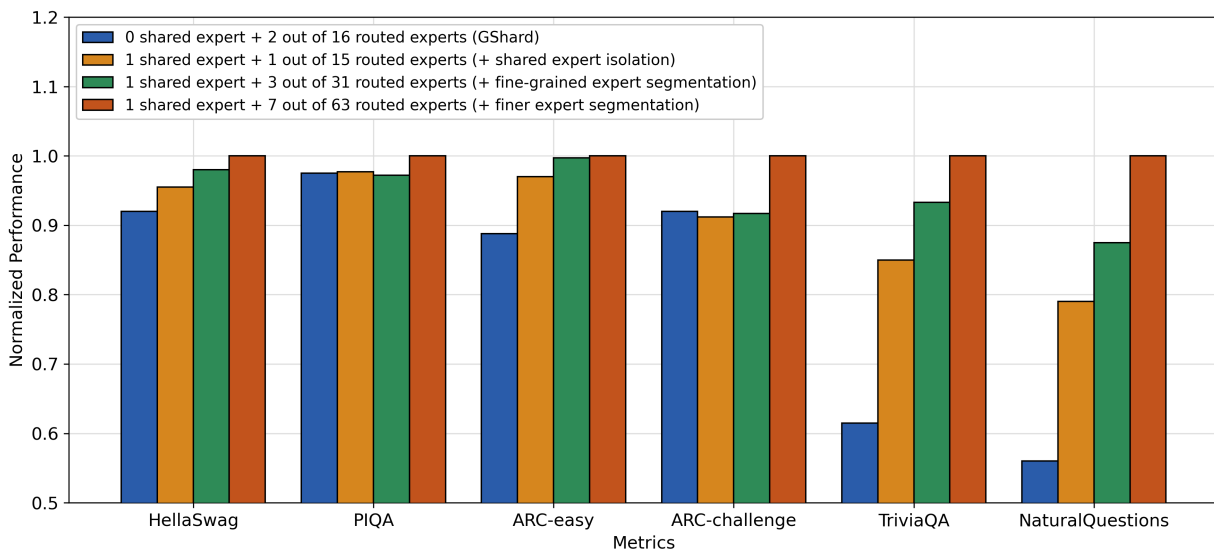
- Mistral, DBRX, DeepSeek v3 softmaxes after the TopK.

Recent variations from DeepSeek and other Chinese LMs



Smaller, larger number of experts + a few shared experts that are always on.

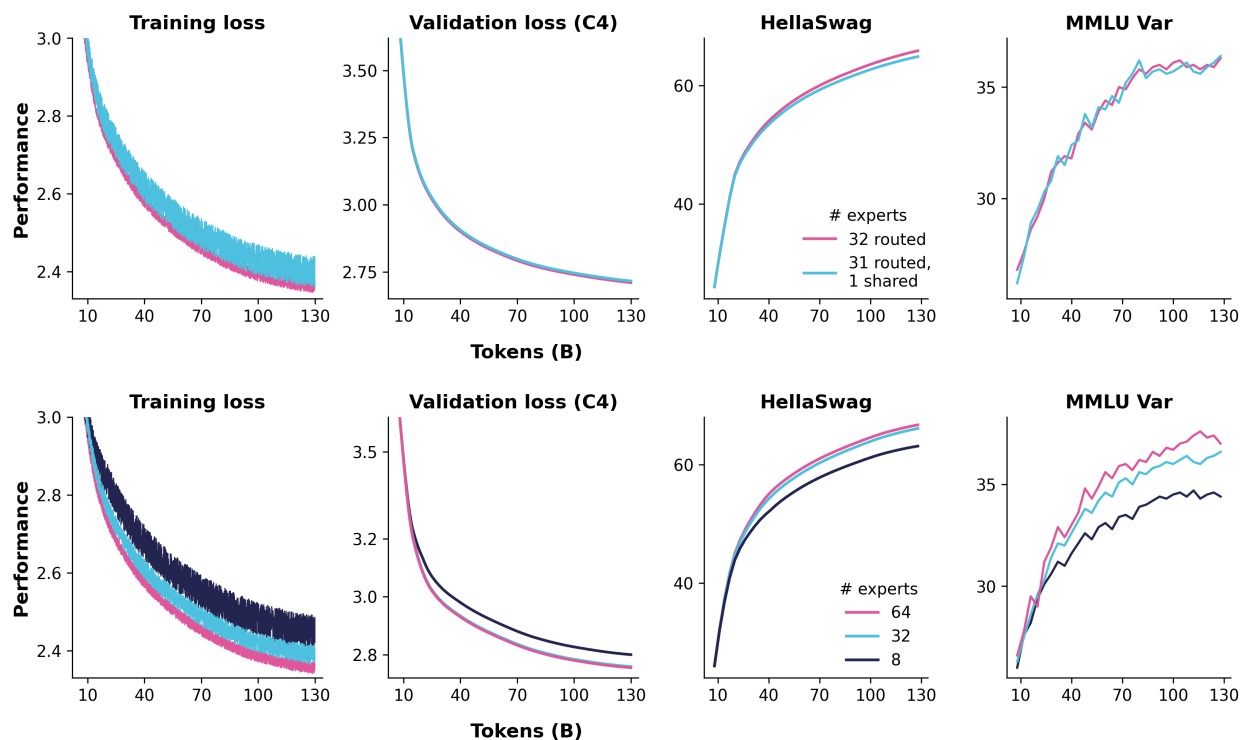
Various ablations from the DeepSeek paper



More experts, shared experts all seem to generally help.

Ablations from O1MoE

Gains from fine-grained experts, none from shared experts.



Expert routing setups for recent MoEs

Model	Routed	Active	Shared	Fine-grained ratio
GShard	2048	2	0	
Switch Transformer	64	1	0	
ST-MOE	64	2	0	
Mixtral	8	2	0	
DBRX	16	4	0	
Grok	8	2	0	
DeepSeek v1	64	6	2	1/4
Qwen 1.5	60	4	4	1/8
DeepSeek v3	256	8	1	1/14
OLMoE	64	8	0	1/8
MiniMax	32	2	0	~1/4
Llama 4 (maverick)	128	1	1	1/2

How to Train our MoEs?

Remark: *We need sparsity for training-time efficiency ... but sparse gating decisions are not differentiable.*

Solutions?

1. Reinforcement learning to optimize gating policies.
2. Stochastic perturbations.
3. Heuristic "balancing" losses.

Guess which one people use in practice?

RL for MoEs

- RL via REINFORCEMENT does work, but not so much better that it's a clear win.
- RL is the "right solution" but gradient variances and complexities means it's not widely used.

Stochastic approximations

$$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k))$$

$$H(x)_i = (x \cdot W_g)_i + \text{StandardNormal}() \cdot \text{Softplus}((x \cdot W_{noise})_i)$$

$$\text{KeepTopK}(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ elements of } v. \\ -\infty & \text{otherwise.} \end{cases}$$

From Shazeer et al 2017 - routing decisions are *stochastic* with Gaussian perturbations.

1. This naturally leads to experts that are a bit more robust.
2. The softmax means that the model learns how to rank k experts.

Stochastic jitters

```

if is_training:
    # Add noise for exploration across experts.
    router_logits += mtf.random_uniform(
        shape=router_logits.shape, minval=1-eps, maxval=1+eps)

    # Convert input to softmax from bfloat16 to float32 for stability.
    router_logits = mtf.to_float32(router_logits)

    # Probabilities for each token.
    router_probs = mtf.softmax(router_logits, axis=-1)

```

Stochastic jitter in Fedus et al 2022. This does uniform multiplicative perturbations for the same goal of getting less brittle experts. This was later removed in Zoph et al 2022.

Heuristic balancing losses

Another key issue - systems efficiency requires that we use experts evenly ...

For each Switch layer, this auxiliary loss is added to the total model loss during training. Given N experts indexed by $i = 1$ to N and a batch $\mathcal{B}$ with T tokens, the auxiliary loss is computed as the scaled dot-product between vectors f and P ,

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i$$

where f_i is the fraction of tokens dispatched to expert i ,

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbf{1}\{\text{argmax } p(x) = i\}$$

and P_i is the fraction of the router probability allocated for expert i ,

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x).$$

The derivative with respect to $p_i(x)$ is

$$\frac{\alpha N}{T^2} \sum \mathbf{1}\{\text{argmax } p(x) = i\},$$

so more frequent use = stronger downweighting.

Example from DeepSeek (v1-2)

Per-expert balancing - same as the above heuristic.

$$\begin{aligned}\mathcal{L}_{\text{ExpBal}} &= \alpha_1 \sum_{i=1}^{N'} f_i P_i, \\ f_i &= \frac{N'}{K'T} \sum_{t=1}^T \mathbf{1}(\text{Token } t \text{ selects Expert } i), \\ P_i &= \frac{1}{T} \sum_{t=1}^T s_{i,t}.\end{aligned}$$

Per-device balancing - the objective above, but aggregated by device.

$$\begin{aligned}\mathcal{L}_{\text{DevBal}} &= \alpha_2 \sum_{i=1}^D f'_i P'_i, \\ f'_i &= \frac{1}{|\epsilon_i|} \sum_{j \in \epsilon_i} f_j, \\ P'_i &= \sum_{j \in \epsilon_i} P_j.\end{aligned}$$

DeepSeek v3 variation - perExpert biases

Set up a per-expert bias (making it more likely to get tokens) and use online learning.

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j \mid 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

They call this ”**auxiliary loss free balancing**” (but the approach is not fully auxiliary loss free ..)

Although DeepSeek-v3 mainly relies on the auxiliary-loss strategy for load balance, to prevent extreme imbalance within any single sequence, we also employ a complementary sequence-wise balance loss:

$$\mathcal{L}_{\text{Bal}} = \alpha \sum_{i=1}^{N_r} f_i P_i,$$

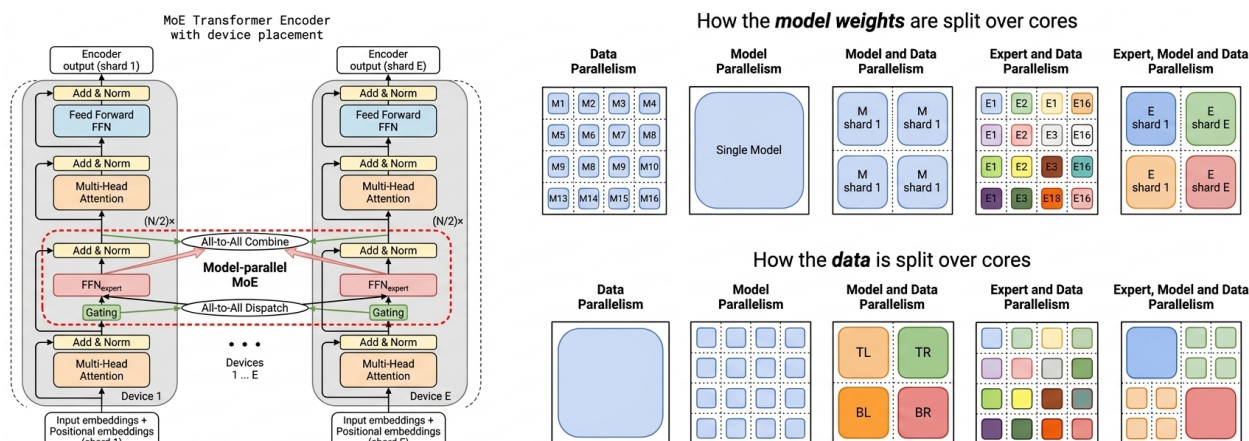
$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^T \mathbf{1}(s_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leq j \neq N_r\}, K_r)),$$

$$s'_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}},$$

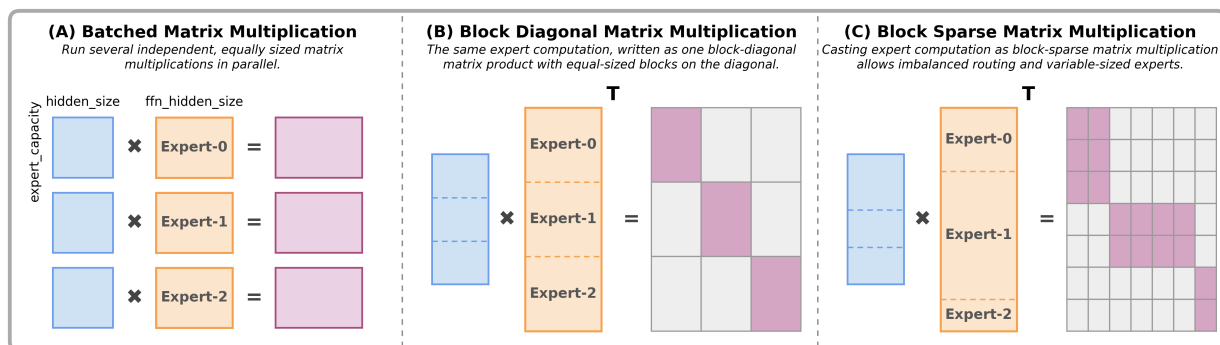
$$P_i = \frac{1}{T} \sum_{t=1}^T s'_{i,t}.$$

Training MoEs - the systems side

MoEs parallelize nicely - each FFN can fit in a device AND enable additional kinds of parallelism.



MoE routing allows for parallelism, but also some complexities.

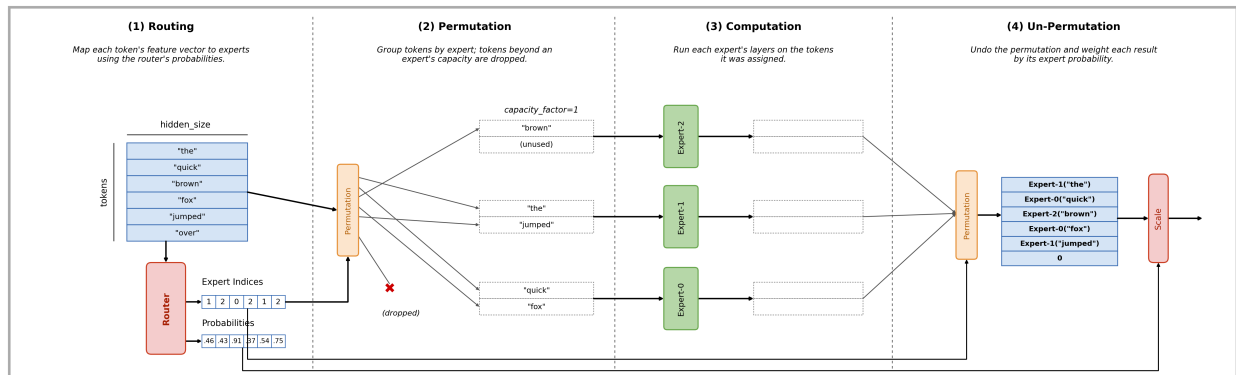


Modern libraries like MegaBlocks (used in many open MoEs) use smarter sparse MatMuls.

Fun side issue - stochasticity of MoE models

MoEs can have additional stochasticity beyond normal models ...

Question: *Why would a MoE have additional randomness?*



Token dropping from routing happens at a *batch* level - this means that other people's queries can drop your token.

Issues with MoEs - fineTuning

- Sparse MoEs can overfit on smaller fine-tuning data.
- Zoph et al solution - finetune nonMoE MLPs.
- DeepSeek solution - use lots of data (1.4M SFT).

Other training methods - upcycling

Question: *Can we use a pretrained LM to initialize an MoE?*

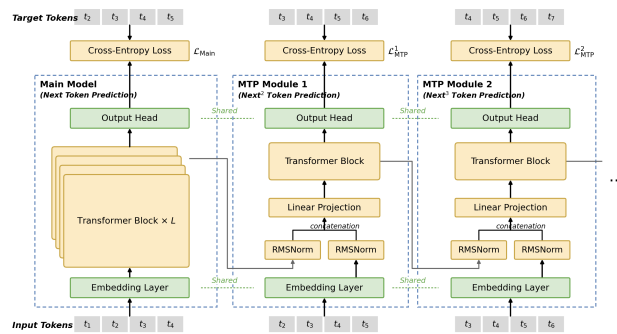
- Use the MiniCPM model ($\text{top}k = 2$, 8 experts, 4B active parameters).
- Simple MoE, shows gains from the base model with 520B tokens for training.
- Initialize from the Qwen 1.8B model ($\text{top}k = 4$, 60 experts with 4 shared).
- Similar architecture to DeepSeek MoE, but one of the first (confirmed) upcycling successes.

- **Complexity:** RoPE conflicts with MLA-style caching (the solution - have a few non-latent key dimensions that can be rotated).

Without RoPE - $\langle Q, K \rangle = \langle hW^Q, W^{UK} \mathbf{c}_t^{KV} \rangle = \langle hW^Q W^{UK}, \mathbf{c}_t^{KV} \rangle$

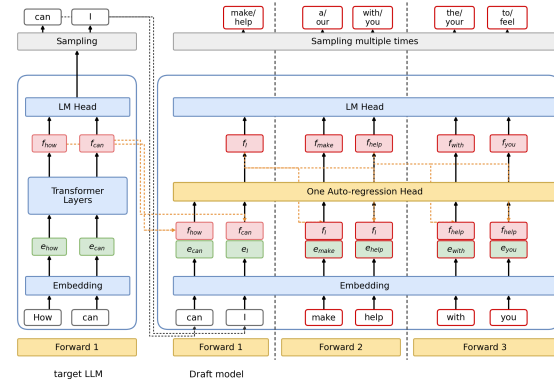
With RoPE - $\langle QR_q, R_k K \rangle = \langle hW^Q R_q, R_k W^{UK} \mathbf{c}_t^{KV} \rangle = \langle hW^Q R_q R_k W^{UK}, \mathbf{c}_t^{KV} \rangle$

MTP: have small, lightweight models that predict multiple steps ahead.



(But they only do MTP with one token ahead)

[Deepseek v3]



[EAGLE]